



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Lossless coding principles

Marco Cagnazzo

Multimedia Communications



Outline

Introduction

Principles of Information Theory

Optimal coding

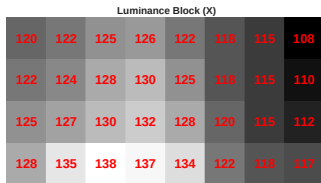
Other Techniques

Neural Lossless Coding

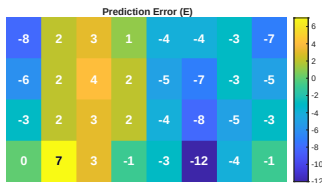
Conclusions



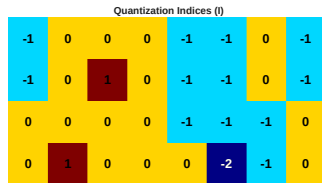
From Samples to Bits: The Encoding Process



(A) Original Luminance $x(n)$



(B) Prediction Error $y(n)$



(C) Quantization Indices $i(n)$

- Spatial Prediction:** Leverages local correlation to reduce variance. $y(n) = x(n) - \hat{x}(n)$
 - Quantization:** Maps the high-precision residuals into a finite set of discrete indices $i(n)$.
- The Central Question: How do we map the resulting indices into an efficient bitstream?

The Answer: **Lossless Coding**

The chosen code must satisfy two fundamental requirements: **Uniquely Decodable:** one-to-one mapping from the bitstream back to the symbol sequence

Compact: smallest possible average codeword length



Variable-Length Coding (VLC)

If we use codewords of different lengths, we face a major hurdle: **The Parsing Problem** How does the decoder "slice" the continuous bitstream into individual symbols if it doesn't know where one ends and the next begins?

Key Questions:

1. **The Theoretical Bound:** How much can we actually gain by using VLC? *Answer:* The limit is defined by the **Source Entropy** $H(X)$.
2. **Practical Implementation:** How do we achieve this limit while ensuring unique decodability? *Answer:* **Huffman** coding achieves the best theoretical performance, but faces scalability problems. **Arithmetic coding** is sub-optimal, but asymptotically optimal and scales easily with the input size
3. All these approaches assume that we know data statistics: what if we don't? *Answer:* Universal coding (LZW)



Choosing a code

Code: $\mathcal{C} : x_i \in \mathcal{X} \rightarrow c_i \in \{0, 1\}^*$

Fixed-length coding (FLC)

- ▶ All codewords have the same length
- ▶ E.g., if $M = 256$ symbols, we need $\lceil \log M \rceil = 8$ bits to encode each symbol
- ▶ For a text, $M = 26$, we need $\lceil \log M \rceil = 5$ bpS (bit per symbol)

Variable-length coding (VLC)

- ▶ ℓ_i : length of codeword c_i
- ▶ We achieve *lossless coding* if:
 - ▶ Decodability condition: prefix condition
 - ▶ Non-equiprobable symbols



Examples of codes

Symbol	Code 1	Code 2	Code 3	Code 4
A	0	0	0	0
B	11	10	01	01
C	111	110	011	11

- ▶ Code 1 is not uniquely decodable : 111111 can be decoded as BBB or CC
- ▶ Code 2 is instantaneously decodable (**prefix condition**)
- ▶ Code 3 is decodable with a delay less than or equal to 1 bit
- ▶ Code 4 is decodable but with undefined delay: 0111111...11 can be decoded as ACCC...CC or BCC...C depending on the number of 1's is even or odd



Examples of codes

Symbol	Prob	Code 1	Code 2	Code 3	Code 4
A	1/2	0	0	0	0
B	1/4	0	1	10	01
C	1/8	1	00	110	011
D	1/8	10	11	111	0111

What is the best code? Go to wooclap.com with the code **GYKKIN**



Examples of codes

Symbol	Prob	Code 1	Code 2	Code 3	Code 4
A	1/2	0	0	0	0
B	1/4	0	1	10	01
C	1/8	1	00	110	011
D	1/8	10	11	111	0111

What is the best code? Go to wooclap.com with the code **GYKKIN**

- ▶ Code 1 is not uniquely decodable A and B have the same codeword and moreover 10 can be decoded in three different ways: CA, CB, D
- ▶ Code 2 is not uniquely decodable since 00 can be AA or C and 11 can be BB or D
- ▶ Code 3 is instantaneously decodable (prefix condition)
- ▶ Code 4 is decodable but with a delay of one bit



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques

Neural Lossless Coding

Conclusions



McMillan's theorem and Kraft's inequality

- ▶ **McMillan's theorem:** Decodable codes do not improve performance with respect to instantaneous codes
We can focus only on instantaneous codes
- ▶ **Kraft's inequality:** there exists an instantaneous code with lengths $\{\ell_1, \dots, \ell_M\}$ **if and only if:**

$$\sum_j 2^{-\ell_j} \leq 1$$

We have a mathematical characterization of instantaneous codes

- ▶ It is a **necessary and sufficient** condition for the existence of a prefix code.
- ▶ If the sum equals 1, the code is said to be **complete** (or maximal).
- ▶ It sets the stage for Shannon's Source Coding Theorem.



Proof of Necessity

Prefix code

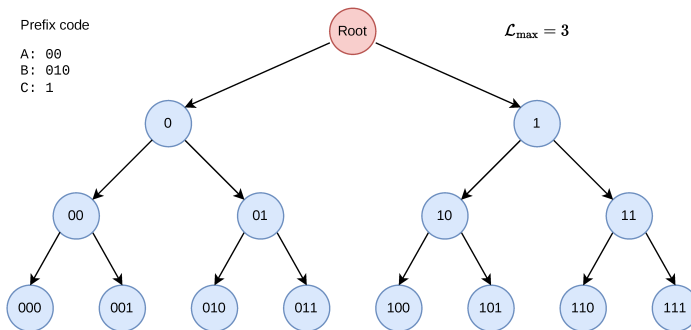
A: 00
 B: 010
 C: 1

$$\mathcal{L}_{\max} = 3$$

Given a prefix code, build a binary tree of depth $L_{\max} = \max(\ell_i)$



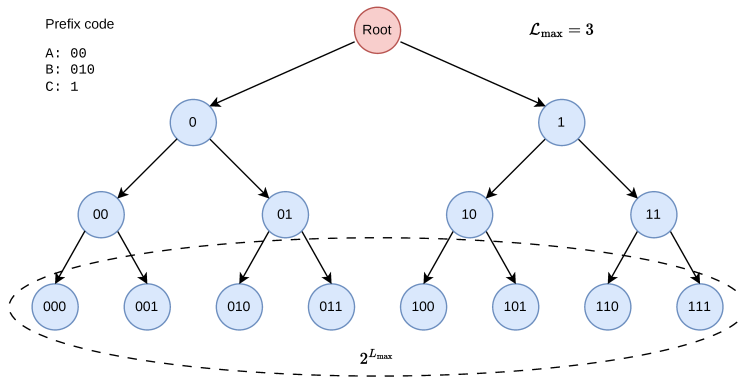
Proof of Necessity



Given a prefix code, build a binary tree of depth $L_{\max} = \max(\ell_i)$



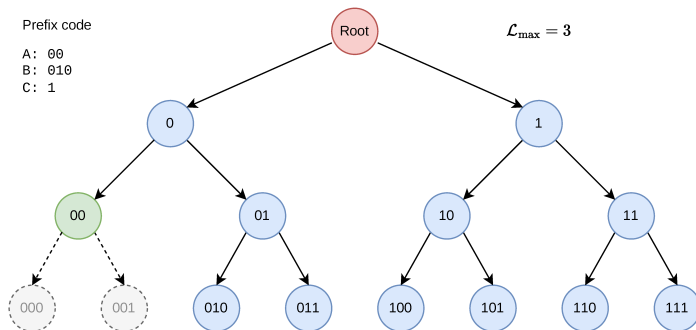
Proof of Necessity



The total number of leaf nodes at level $L_{\max}$ is $2^{L_{\max}}$



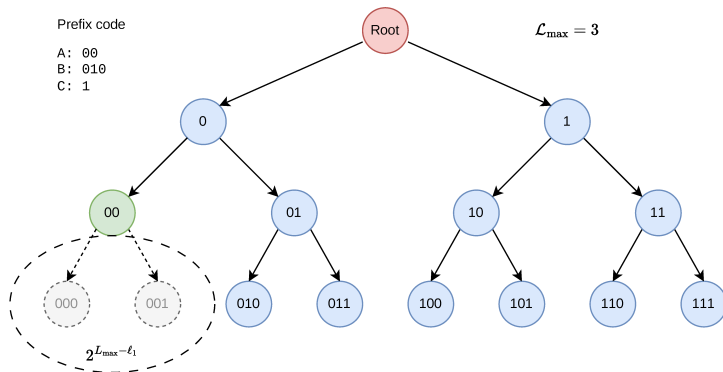
Proof of Necessity



Each codeword c_i of length ℓ_i "occupies" a subtree



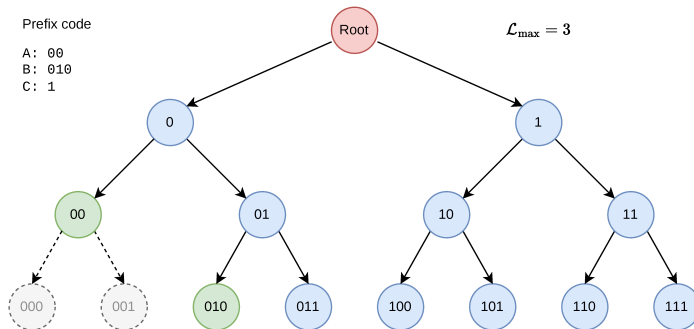
Proof of Necessity



The number of descendant leaves at level $L_{\max}$ "blocked" by c_i is: $\text{Leaves}(c_i) = 2^{L_{\max}-\ell_i}$



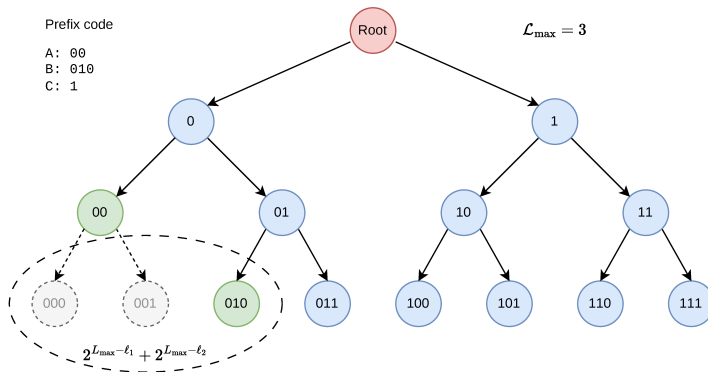
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



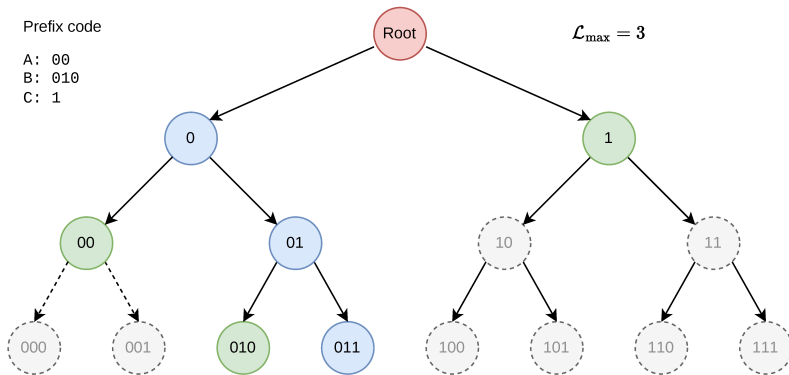
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



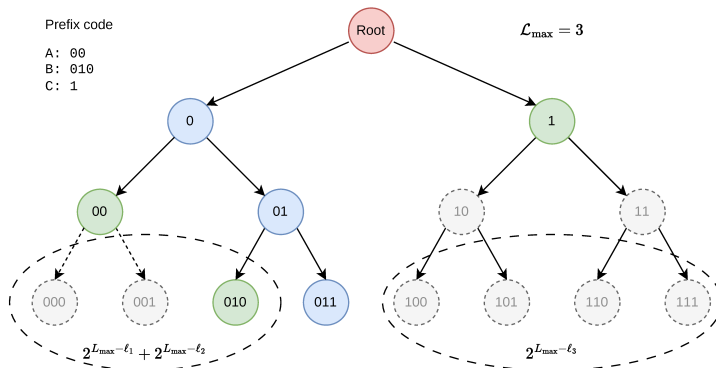
Proof of Necessity



Prefix Property: No codeword is a descendant of another, thus these sets of leaves are **disjoint**



Proof of Necessity



The total number of blocked leaves cannot exceed the available leaves: $\sum_{i=1}^N 2^{L_{\max}-\ell_i} \leq 2^{L_{\max}}$

Dividing by $2^{L_{\max}}$ yields the Kraft inequality: $\sum_{i=1}^N 2^{-\ell_i} \leq 1$



Proof of Sufficiency

Goal: Given $\{\ell_i\}$ such that $\sum 2^{-\ell_i} \leq 1$, construct a prefix code.

Greedy Construction Algorithm:

1. Sort lengths: $\ell_1 \leq \ell_2 \leq \dots \leq \ell_N$.
2. At each step k , pick an available node at depth ℓ_k .
3. Mark all its descendants as "forbidden".



Proof of Sufficiency

Goal: Given $\{\ell_i\}$ such that $\sum 2^{-\ell_i} \leq 1$, construct a prefix code.

Greedy Construction Algorithm:

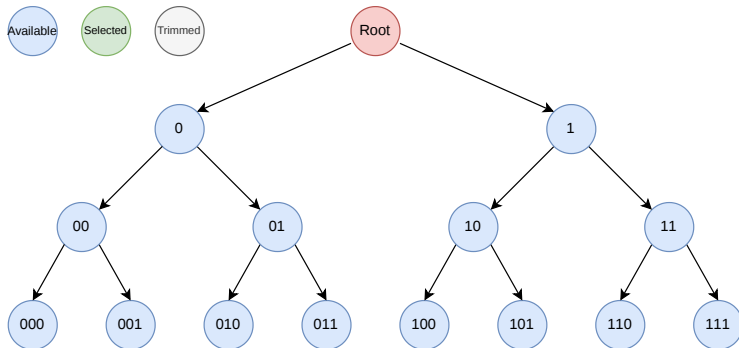
1. Sort lengths: $\ell_1 \leq \ell_2 \leq \dots \leq \ell_N$.
2. At each step k , pick an available node at depth ℓ_k .
3. Mark all its descendants as "forbidden".

Existence Argument: At step k , the "consumed capacity" of the tree is $\sum_{i=1}^{k-1} 2^{-\ell_i}$. Since the total sum is ≤ 1 , and $\ell_k \geq \ell_{k-1}$, there must be at least one branch available at depth ℓ_k to place the next codeword.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$

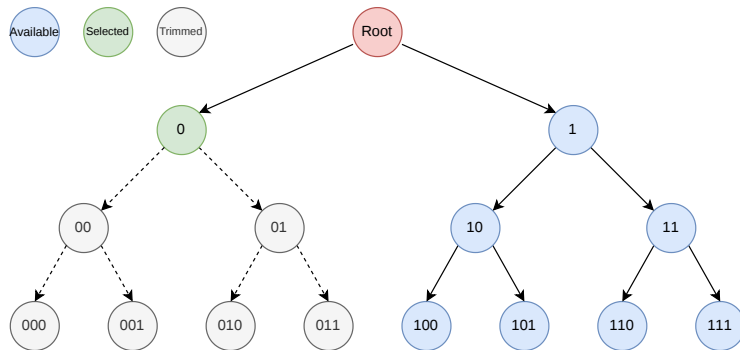


Build a complete binary tree, depth equal to $L_{\max}$



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



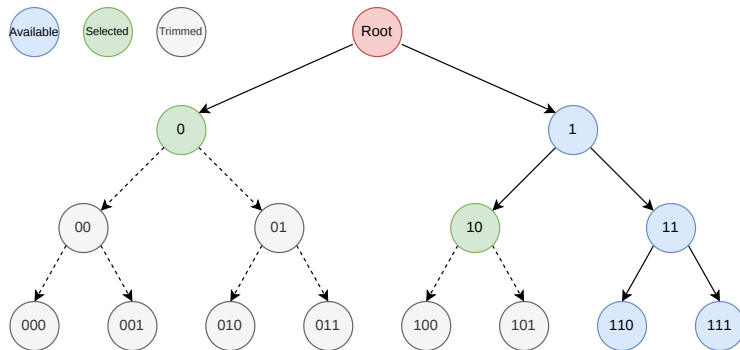
Select a node at depth ℓ_1 , trim its subtree

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



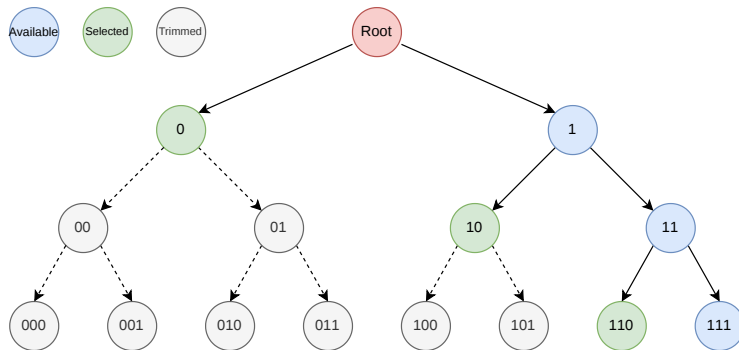
Select an available node at depth ℓ_2 , trim its subtree

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



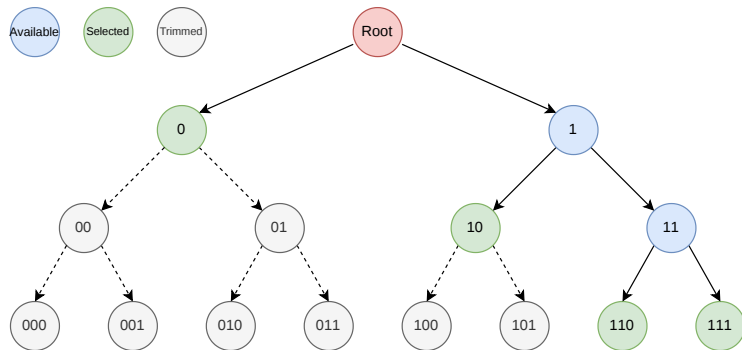
Select an available leaf

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Example: building a prefix code

Let us build a code with lengths $\{1, 2, 3, 3\}$: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1$



Select an available leaf.
The code is $\{0, 10, 110, 111\}$

Conclusion: Lengths satisfying Kraft's inequality $\Rightarrow$ we can build a prefix code with those lengths.



Information and entropy

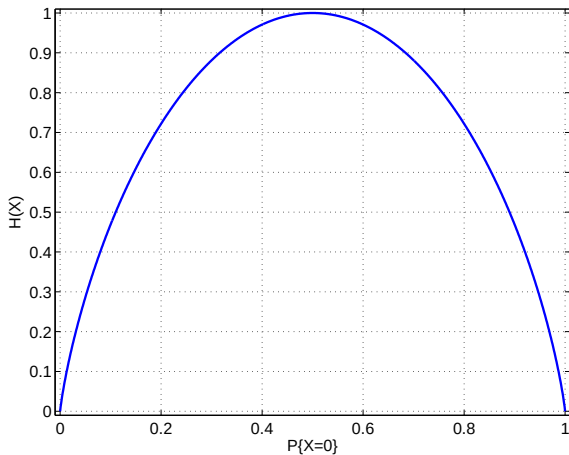
- ▶ RV model of the information source: X
- ▶ Symbol x_i has probability p_i
- ▶ Information associated to x_i is $I(x_i) = -\log p_i$
 - ▶ $I(x_i) \geq 0$
 - ▶ If $p_i = 1$, $I = 0$
 - ▶ If two symbols are independent, $I(x_i, x_j) = I(x_i) + I(x_j)$
- ▶ Source entropy: average symbol information

$$H(X) = - \sum_i p_i \log p_i$$

- ▶ Interpretation:
 - ▶ Average uncertainty associated to the random variable X



Binary variable entropy



Case of binary RV

Alphabet: $\{0, 1\}$

The probability mass function (pmf) is given by:

$$p = P\{X = 0\}$$

$$q = P\{X = 1\} = 1 - p \quad \text{as a consequence}$$

$$H(X) = -p \log(p) - (1 - p) \log(1 - p)$$

The entropy can be traced as a function of a single parameter p



Distribution with maximum entropy

Constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \quad \sum_{i=1}^M p_i = 1$$

In order to solve this **constrained optimization problem**, we need a specific tool: the Lagrange's method (or method of Lagrange's multipliers)



Constrained optimization: Lagrange's method

Let's consider a function $f : x \in \mathbb{R}^n \rightarrow \mathbb{R}$

In order to find the maximum or minimum of a function $f(x)$ subjected to the equality constraint $\phi(x) = 0$, we look for the stationary points of the functional

$$J(x, \lambda) = f(x) + \lambda \cdot \phi(x)$$

with $\lambda \in \mathbb{R}$. The solution is among these points.

The stationary points are computed by setting to zero all the derivatives of J :

$$\frac{\partial J}{\partial x_i} = 0 \quad \frac{\partial J}{\partial \lambda} = 0$$



Distribution with maximum entropy

Let us go back to our constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \quad \sum_{i=1}^M p_i = 1$$

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method:



Distribution with maximum entropy

Let us go back to our constrained maximization problem:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i} \qquad \sum_{i=1}^M p_i = 1$$

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method: $p^* = [p_1^* \ p_2^* \ \dots \ p_M^*]$ such that $p_i^* = \frac{1}{M} \quad \forall i \in \{1, 2, \dots, M\}$

$$J(p) = - \sum_{i=1}^M p_i \log p_i + \lambda \left(\sum_{i=1}^M p_i - 1 \right) \qquad \frac{\partial J}{\partial p_i}(p^*) = 0$$

$$\frac{\partial J}{\partial p_i} = - \left(\frac{\log e}{p_i} p_i + \log p_i \right) + \lambda \qquad \log p_i^* = \lambda - \log e = \text{constant}$$



Joint entropy

- ▶ For a couple of r.v.'s X and Y
- ▶ Joint probability distribution $p_{i,j} = P\{X = x_i, Y = y_j\}$
- ▶ Joint entropy: average information of couples

$$H(X, Y) = - \sum_{i,j} p_{i,j} \log p_{i,j}$$

- ▶ Formally, there is no difference between the joint probability of a couple X, Y and the entropy of a single r.v. having the same set of probabilities
- ▶ The entropy does not depends on the values, only on the probabilities



Conditional entropy

- ▶ We consider a couple of r.v. X and Y
- ▶ Let $p_j = P\{Y = y_j\}$
- ▶ Conditional entropy:

$$H(X|Y) = \sum_j p_j H(X|Y = y_j)$$

- ▶ It is easy to show that:

$$\begin{aligned}
 H(X, Y) &= H(Y) + H(X|Y) \\
 &= H(X) + H(Y|X)
 \end{aligned}$$



Properties

$$H(X) \geq 0$$

$$H(X, Y) = H(Y) + H(X|Y)$$

$$H(X) + H(Y|X)$$

$$H(X, Y) \leq H(X) + H(Y)$$

$$H(X|Y) \leq H(X)$$

$$H(X) \leq \log_2 M$$

with equality $\Leftrightarrow$ independence

with equality $\Leftrightarrow$ independence

with equality $\Leftrightarrow X \sim \mathcal{U}$



Conditional Entropy

Formal Definition

- ▶ The average uncertainty about X given Y : $H(X|Y) = \sum_j p_j H(X|Y = y_j)$
- ▶ **The Chain Rule:**
 $H(X, Y) = H(Y) + H(X|Y)$

What does it represent?

- ▶ **Residual Uncertainty:** The amount of information still required to describe X after Y has been observed.
- ▶ **Equivocation:** In communications, it is the information lost due to a noisy channel.

If X and Y are independent, $H(X|Y) = H(X)$.

If X is a deterministic function of Y , $H(X|Y) = 0$.



Properties

Fundamental Property: Conditioning Reduces Entropy

$$H(X|Y) \leq H(X)$$

"Information never hurts": knowing Y can only decrease (or leave unchanged) the uncertainty about X .



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:
 - ▶ $\forall i = 1, \dots, M, \ell_i \in \mathbb{N}$



Optimal code

- ▶ Problem formulation
- ▶ Given a set of M symbols with probabilities p_i
- ▶ Find the set of M lengths ℓ_i such that:
 - ▶ $\forall i = 1, \dots, M, \ell_i \in \mathbb{N}$
 - ▶ The Kraft's inequality is satisfied (prefix code)



Optimal code

- ▶ But we need to drop condition $\ell_i \in \mathbb{N}$
- ▶ Constrained minimization:

$$\underline{\ell}^* = \arg \min_{\underline{\ell}} \sum_i p_i \ell_i$$

$$\text{subject to } \sum_i 2^{-\ell_i} = 1$$



Optimal code

- ▶ But we need to drop condition $\ell_i \in \mathbb{N}$
- ▶ Constrained minimization:

$$\underline{\ell}^* = \arg \min_{\underline{\ell}} \sum_i p_i \ell_i$$

$$\text{subject to } \sum_i 2^{-\ell_i} = 1$$

$$J(\underline{\ell}) = \sum_i p_i \ell_i + \lambda \left(\sum_i 2^{-\ell_i} - 1 \right)$$

$$\frac{\partial J}{\partial \ell_i} = p_i - (\lambda \ln 2) 2^{-\ell_i^*} = 0$$

$$\sum_i p_i = (\lambda \ln 2) \sum_i 2^{-\ell_i^*}$$

$$1 = \lambda \ln 2$$

$$2^{-\ell_i^*} = p_i$$

$$\ell_i^* = -\log_2 p_i$$

$$\mathcal{L}^* = \sum_i -p_i \log_2 p_i = H(X)$$



Why the Equality in Kraft's Constraint?

To minimize $L = \sum p_i \ell_i$, we must use the available "prefix space" efficiently.

The "Unused Branch" Argument:

- ▶ If $\sum 2^{-\ell_i} < 1$, there exists at least one leaf in the binary tree that is neither a codeword nor a descendant of one.
- ▶ We could **shorten** some codewords by moving them "up" the tree.
- ▶ Shortening ℓ_i strictly decreases L (since $p_i > 0$).

Conclusion: The minimum average length L^* must satisfy the **active constraint**:

$$\sum 2^{-\ell_i^*} = 1$$

This represents a **Complete Code**: an efficient tree with no "wasted" branches.



Shannon's source coding theorem

The minimum average length $\mathcal{L}^* = H(X)$ can be achieved if and only if the lengths are *dyadic*:

$$\forall i \in \{1, 2, \dots, M\}, \exists k \in \mathbb{N} \mid p_i = 2^{-k}$$

Therefore, in the general case, the average length can only be greater than or equal to the entropy:

$$\mathcal{L}^* \geq H(X)$$

with equality if and only if the lengths are dyadic

This is the **Shannon's Source Coding Theorem**



Entropy coding

Shannon's theorem:

Optimal coding rate $\geq$ source entropy

therefore optimal coding is called *Entropy coding*.

- ▶ It is a strict identity if the probability distribution is *dyadic*, i.e.,
 $\forall i, \exists k \in \mathbb{N} : p_i = 2^{-k}$
- ▶ However, when M increases, the optimal coding rate is practically very close to H



Entropy coding

- ▶ It easy to find an instantaneous code such that $\mathcal{L} < H(X) + 1$, we just take $\ell_i = \lceil -\log_2 p_i \rceil$
- ▶ Kraft's inequality is satisfied:

$$\ell_i = -\log p_i + \delta_i$$

$$0 \leq \delta_i < 1$$

$$2^{-\ell_i} = p_i \times 2^{-\delta_i} = \epsilon_i p_i$$

$$\frac{1}{2} < \epsilon_i \leq 1$$

$$\sum_i 2^{-\ell_i} = \sum_i \epsilon_i p_i \leq \sum_i p_i = 1$$

- ▶ The average length is obtained as follows:

$$\ell_i = \lceil -\log_2 p_i \rceil < -\log_2 p_i + 1$$

$$p_i \ell_i < -p_i \log_2 p_i + p_i$$

$$\sum_i p_i \ell_i < \sum_i (-p_i \log_2 p_i + p_i)$$

$$\mathcal{L} < H(X) + 1$$

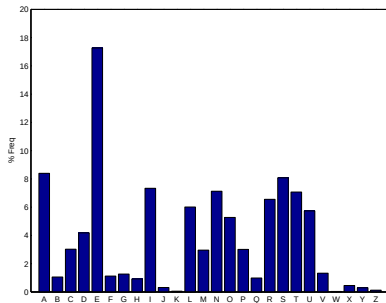
- ▶ Since $\mathcal{L}^* \leq \mathcal{L}$,

$$\boxed{H(X) \leq \mathcal{L}^* < H(X) + 1}$$



Example: French text compression

Source entropy	3.999 bpS
Method	Any VLC
Coding rate ($\mathcal{L}$)	≥ 3.999 bpS
Compression ratio	≤ 1.25



Letter distribution in a French text



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques

Neural Lossless Coding

Conclusions



Huffman coding

Huffman discovered how to build the optimal lossless coder for any source with known probabilities. **Steps:**

1. Create leaf nodes for each symbol, weighted by p_i
2. While there's more than one node:
 - ▶ Select two nodes with lowest weights
 - ▶ Create a new internal node as their parent
 - ▶ Set new node's weight as sum of children's weights
3. Assign '0' to left edges, '1' to right edges
4. Read code for each symbol from root to leaf



Huffman coding

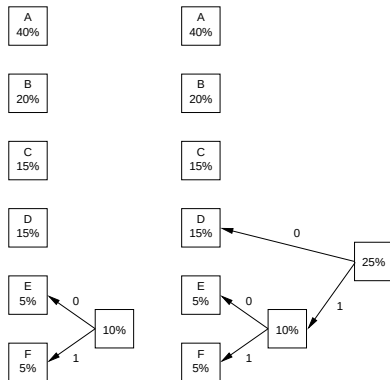
Example:

Symbol	Probability
A	0.4
B	0.2
C	0.15
D	0.15
E	0.05
F	0.05

6 symbols, 3 bits per symbol (without coding)

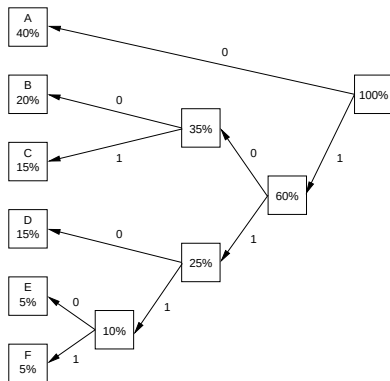


Huffman coding





Huffman coding





Huffman coding

Symbol	Probability	Codeword
A	0.4	0
B	0.2	100
C	0.15	101
D	0.15	110
E	0.05	1110
F	0.05	1111

Find $\mathcal{L}$ and H



Huffman coding

Symbol	Probability	Codeword
A	0.4	0
B	0.2	100
C	0.15	101
D	0.15	110
E	0.05	1110
F	0.05	1111

Find $\mathcal{L}$ and H

$$\mathcal{L} = 0.4 \cdot 1 + 0.2 \cdot 3 + 0.15 \cdot 3 + 0.15 \cdot 3 + 0.05 \cdot 4 + 0.05 \cdot 4 = 2.3 \text{ bits/Symbol}$$

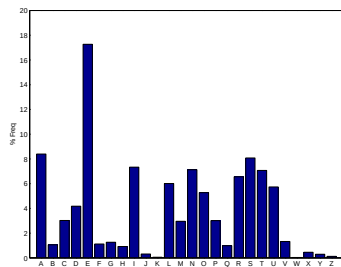
$$\begin{aligned}
 H &= 0.4 \cdot \log_2 \frac{1}{0.4} + 0.2 \cdot \log_2 \frac{1}{0.2} + 2 \cdot 0.15 \cdot \log_2 \frac{1}{0.15} + 2 \cdot 0.05 \cdot \log_2 \frac{1}{0.05} \\
 &\cong 2.2464 \text{ bits/Symbol}
 \end{aligned}$$

Go to wooclap.com with the code **GYKKIN**

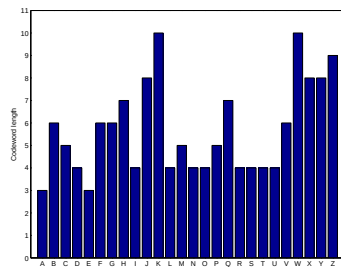


Example: Compression of French text

Method	Huffman
Source entropy	3.999 bpS
Coding rate ($\mathcal{L}$)	4.041 bpS
Compression ratio	1.238



Letter probabilities in a French text



Huffman (i.e., optimal) codeword lengths



Huffman coding

- ▶ Example: coding a scanned document. Typically many pixels are white and only few are black. If X is a r.v. representing the color of a pixel:
- ▶ $\Pr(\{X = B\}) = p \ll 1$
- ▶ $\Pr(\{X = W\}) = 1 - p$
- ▶ $H(X) \ll 1$
- ▶ Huffman coding: $B \rightarrow 0 \quad W \rightarrow 1$
- ▶ $\mathcal{L} = 1 \gg H(X)$
- ▶ Conclusion: the entropy is very small, but the optimal code has a coding rate much larger than the entropy: can we do better?



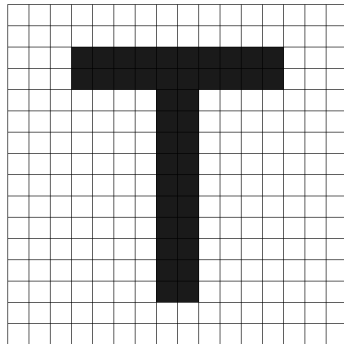
Huffman coding

$$H(X) = 0.586$$

Huffman code:

■	0
□	1

Code length: 1 bpp

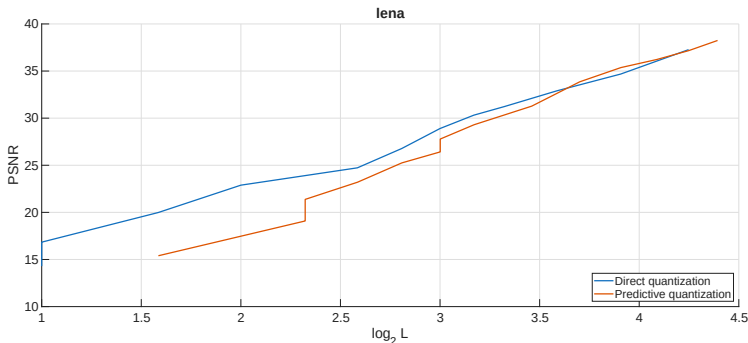


$$P(\square) = 86.7\%$$

$$P(\blacksquare) = 13.3\%$$



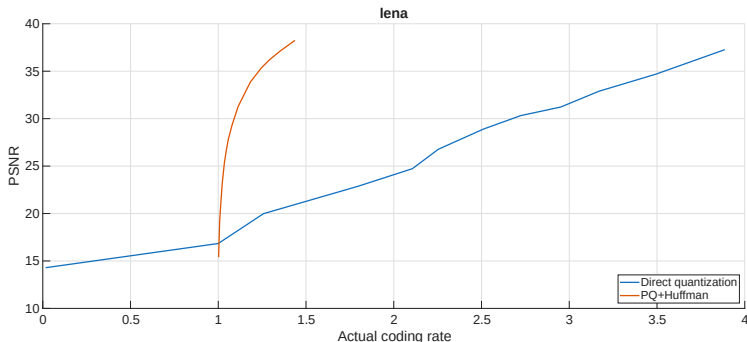
Huffman Coding and Predictive Quantization



Quantized prediction errors encoded with **FLC** (Fixed-Length)



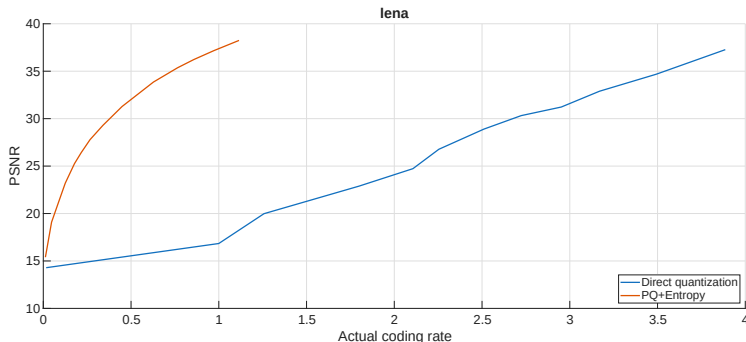
Huffman Coding and Predictive Quantization



Quantized prediction errors encoded with **Huffman** (VLC)



Huffman Coding and Predictive Quantization



Theoretical limit: **Entropy** of quantized prediction errors



Huffman coding

How to improve performance?

- ▶ Let us refer to the block of K symbols $X^K = (X_1 X_2 \dots X_K)$:

$$H[(X_1 X_2 \dots X_K)] \leq \sum_{i=1}^K H(X_i)$$

with equality if and only if components of X^K are **independent** each from the other

- ▶ Idea of **block coding**: Encode the block X^K rather than each individual component
 1. If the symbols are not independent, $H(X^K)/K < H(X_i)$
 2. $\mathcal{L} < H(X^K) + 1 \Leftrightarrow \mathcal{L}_S < H(X^K)/K + 1/K$

If the symbols are identically distributed (but not necessarily independent),
 $\mathcal{L}_S < H(X^K)/K + 1/K \leq H(X_i) + 1/K$



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels
- ▶ $\Pr(BB) = p^2$
- ▶ $\Pr(WB) = p(1 - p) \approx p$
- ▶ $\Pr(BW) = p(1 - p) \approx p$
- ▶ $\Pr(WW) = (1 - p)^2 \approx 1 - 2p$
- ▶ $H(X_1, X_2) = 2H(X) \ll 2$
- ▶ Huffman coding:



Huffman coding

Block coding

- ▶ Example: coding a B/W image with independent pixels
- ▶ $\Pr(BB) = p^2$
- ▶ $\Pr(WB) = p(1 - p) \approx p$
- ▶ $\Pr(BW) = p(1 - p) \approx p$
- ▶ $\Pr(WW) = (1 - p)^2 \approx 1 - 2p$
- ▶ $H(X_1, X_2) = 2H(X) \ll 2$
- ▶ Huffman coding: $WW \rightarrow 0 \quad WB \rightarrow 10 \quad BW \rightarrow 110 \quad BB \rightarrow 111$
- ▶ $\mathcal{L} \approx 1$
- ▶ $\mathcal{L}_S \approx \frac{1}{2}$



Huffman coding

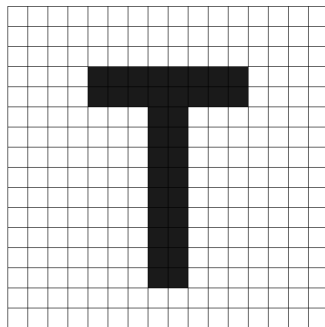
$$H([X_1 X_2]) = 1.022$$

$$H([X_1 X_2])/2 = 0.511$$

Huffman code:

■ ■	00
□ ■	010
■ □	011
□ □	1

Code length: 1.3 bits per block, **0.65 bpp**



$$P(\square \square) = 80.0\%$$

$$P(\square \blacksquare) = 5.00\%$$

$$P(\blacksquare \square) = 5.00\%$$

$$P(\blacksquare \blacksquare) = 10.0\%$$

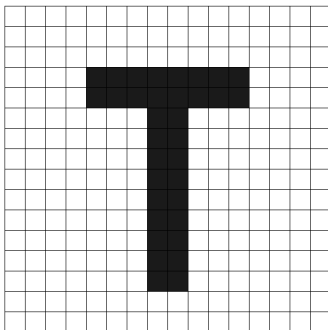


Huffman coding

$$H([X_1 X_2 X_3 X_4]) = 1.533$$

$$H([X_1 X_2 X_3 X_4])/4 = 0.383$$

Code length: 1.733 bits
per block, **0.433** bpp



$$P(\begin{smallmatrix} \square & \square \\ \square & \square \end{smallmatrix}) = 74.7\%$$

$$P(\blacksquare) = 7.56\%$$

$$P(\begin{smallmatrix} \square & \square \\ \blacksquare & \square \end{smallmatrix}) = 0.89\%$$

$$P(\begin{smallmatrix} \square & \square \\ \square & \blacksquare \end{smallmatrix}) = 0.89\%$$

$$P(\begin{smallmatrix} \square & \square \\ \blacksquare & \blacksquare \end{smallmatrix}) = 0.44\%$$

$$P(\begin{smallmatrix} \square & \blacksquare \\ \square & \square \end{smallmatrix}) = 0.44\%$$

$$P(\begin{smallmatrix} \square & \blacksquare \\ \blacksquare & \square \end{smallmatrix}) = 3.11\%$$

$$P(\begin{smallmatrix} \square & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}) = 2.22\%$$

$$P(\begin{smallmatrix} \blacksquare & \square \\ \square & \square \end{smallmatrix}) = 4.44\%$$

$$P(\begin{smallmatrix} \blacksquare & \square \\ \square & \blacksquare \end{smallmatrix}) = 4.44\%$$

$$P(\begin{smallmatrix} \blacksquare & \square \\ \blacksquare & \square \end{smallmatrix}) = 0.44\%$$

$$P(\begin{smallmatrix} \blacksquare & \blacksquare \\ \square & \square \end{smallmatrix}) = 0.44\%$$

$$P(\begin{smallmatrix} \blacksquare & \blacksquare \\ \square & \square \end{smallmatrix}) = 0.00\%$$

$$P(\begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \square \end{smallmatrix}) = 0.00\%$$

$$P(\begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}) = 0.00\%$$

$$P(\begin{smallmatrix} \blacksquare & \square \\ \blacksquare & \blacksquare \end{smallmatrix}) = 0.00\%$$



Huffman coding

Block coding

- ▶ Hypothesis: $\frac{H(X^K)}{K}$ is converging
- ▶ This is true for example in the case of stationary process
- ▶ Average lenght of optimal code:

$$\begin{aligned}
 H(X^K) &\leq \mathcal{L}^* < H(X^K) + 1 \\
 \frac{H(X^K)}{K} &\leq \frac{\mathcal{L}^*}{K} = \mathcal{L}_S^* < \frac{H(X^K)}{K} + \frac{1}{K} \\
 \lim_K \mathcal{L}_S^* &= \lim_K \frac{H(X^K)}{K} = \mathcal{H}(X) \\
 \mathcal{L}_S^* &\rightarrow \mathcal{H}(X) \leq H(X)
 \end{aligned}$$

- ▶ $\mathcal{H}(X)$ is called *entropic rate*



Huffman coding

Block coding

- ▶ Block coding:

$$\mathcal{L}_S^* \rightarrow \mathcal{H}(X) \leq H(X)$$

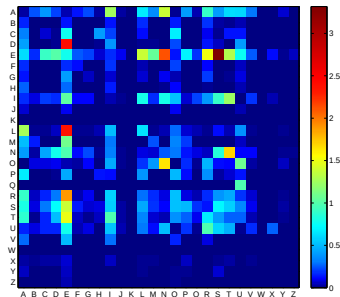
- ▶ The best performance is attained when the whole message of K symbols is encoded as a single block
- ▶ This means an alphabet of N^K symbols
- ▶ Block coding improves performance even for iid variables, since it asymptotically removes the 1-bit penalty for non-dyadic distributions



Example: Compression of French text

K=1	Letter Entropy	3.999 bpB	3.999 bpS
K=2	Digram Entropy	7.440 bpB	3.720 bpS
K=3	Trigrams Entropy	9.452 bpB	3.151 bpS

bpB: bits per block; bpS: bits per symbol, bits per letter



Digram distribution in a French text

Most common trigrams:

ait	ent	les
1.59%	1.25%	0.94%
lle	des	ant
0.78%	0.72%	0.70%
que	our	ien
0.67%	0.63%	0.60%



Limits of Huffman coding

We would like to use Huffman with larger and larger K

This is practically impossible

- ▶ Complexity is exponential with K
- ▶ The joint probability estimation for large blocks is costly and unreliable



Limits of Huffman coding

We would like to use Huffman with larger and larger K

This is practically impossible

- ▶ Complexity is exponential with K
- ▶ The joint probability estimation for large blocks is costly and unreliable

Arithmetic coding solves (at least) the first problem



Arithmetic coding

- ▶ *Arithmetic coding* allows to perform block coding or context-based coding with linear complexity
- ▶ Instead of producing the entire dictionary and then selecting the codeword, only the codeword associated to the block of n input symbols is built
- ▶ The arithmetic is suboptimal, but *asymptotically* optimal

$$\mathcal{L} \leq H(X^K) + 2$$

$$\mathcal{L}_S = \mathcal{L}/K$$

$$\lim_{K \rightarrow \infty} \mathcal{L}_S = \lim_{K \rightarrow \infty} \frac{H^K}{K} = \mathcal{H}(X)$$

- ▶ Low-complexity encoding and decoding: two sums and two multiplications per input symbols (arithmetic operations)

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: **ACFD**

► Start: $[0, 1)$



Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: **ACFD**



► **Start:** $[0, 1)$

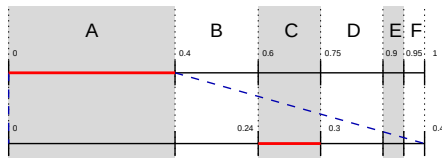
► **A:** $[0, 0.4)$

Center: 0.2. Precision: $p_1/2 = 0.2$.

$L(1) = -\lceil \log_2 p_1 \rceil + 1$

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

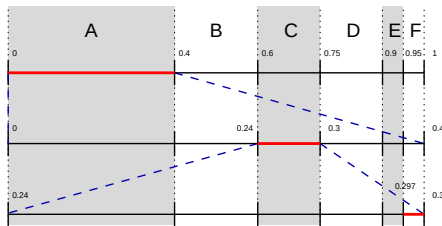
Input sequence: **ACFD**



- **Start:** $[0, 1)$
- **A:** $[0, 0.4)$
Center: 0.2. Precision: $p_1/2 = 0.2$.
 $L(1) = -\lceil \log_2 p_1 \rceil + 1$
- **C:** $[0.24, 0.30)$
Center: 0.275. Precision: $p_1 p_3 / 2$.
 $L(2) = -\lceil \log_2 p_1 + \log_2 p_3 \rceil + 1$

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

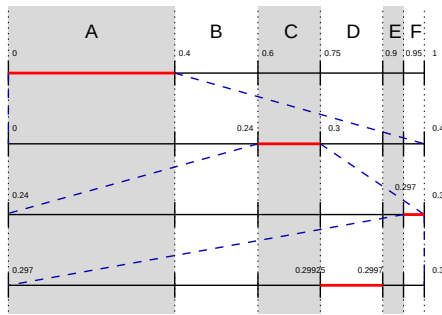
Input sequence: **ACFD**



- ▶ **Start:** $[0, 1)$
- ▶ **A:** $[0, 0.4)$
Center: 0.2. Precision: $p_1/2 = 0.2$.
 $L(1) = -\lceil \log_2 p_1 \rceil + 1$
- ▶ **C:** $[0.24, 0.30)$
Center: 0.275. Precision: $p_1 p_3/2$.
 $L(2) = -\lceil \log_2 p_1 + \log_2 p_3 \rceil + 1$
- ▶ **F:** $[0.297, 0.30)$
Center: 0.2985. Precision: $p_1 p_3 p_6/2$.
 $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 \rceil + 1$

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: **ACFD**



- ▶ **Start:** $[0, 1)$
- ▶ **A:** $[0, 0.4)$
Center: 0.2. Precision: $p_1/2 = 0.2$.
 $L(1) = -\lceil \log_2 p_1 \rceil + 1$
- ▶ **C:** $[0.24, 0.30)$
Center: 0.275. Precision: $p_1 p_3/2$.
 $L(2) = -\lceil \log_2 p_1 + \log_2 p_3 \rceil + 1$
- ▶ **F:** $[0.297, 0.30)$
Center: 0.2985. Precision: $p_1 p_3 p_6/2$.
 $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 \rceil + 1$
- ▶ **D:** $[0.29925, 0.2997)$
Center: 0.299475. Precision: $p_1 p_3 p_6 p_4/2$. $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 + \log_2 p_4 \rceil + 1$



Arithmetic coding: average length

$$L(n) = - \left\lceil \sum_{i=1}^n \log_2 p(x_i) \right\rceil + 1 < - \sum_{i=1}^n \log_2 p(x_i) + 2$$

$$\bar{L}(n) < \frac{- \sum_{i=1}^n \log_2 p(x_i) + 2}{n}$$

$$\mathcal{L} = E[\bar{L}(n)] < \frac{- \sum_{i=1}^n E[\log_2 p(x_i)] + 2}{n}$$

$$\mathcal{L} < H(X) + \frac{2}{n} \rightarrow H(X)$$



Arithmetic coding

- ▶ Problem: estimation of $P(X^K)$, where K can be as large as the size of the message
- ▶ In the previous exemple, symbols were supposed independent,

$$P(X^K) = \prod_{i=1}^K p(x_i)$$
- ▶ Symbols statistics can be learned during the encoding process (adaptivity): we store an occurrence count vector, and we update (at the encoder and the decoder) according to some agreed rule



Context-based coding

- ▶ Estimation of joint probability $P(X^K)$ may be difficult
- ▶ On the other hand, often a symbol only depends on a few neighbors
- ▶ We define as *context* a set of a few, let us say N_S previous symbols that mainly impacts on the current one
- ▶ The number of possible contexts is $N_C = M^{N_S}$ at most
 - ▶ Contexts may be clustered in order to reduce complexity and improve probability estimation
- ▶ Using N_C contexts is equivalent to having N_C different arithmetic encoders and switching from the one to the other



Context-based coding

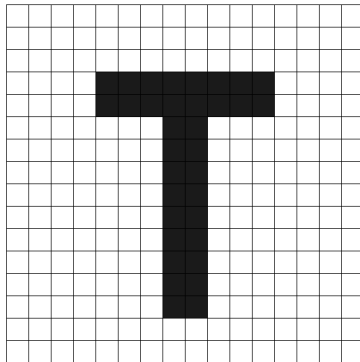
$$H(X|\square) = 0.322$$

$$H(X|\blacksquare) = 0.918$$

$$H(X|Y) = 0.332P(\square) +$$

$$+0.918P(\blacksquare) = 0.406$$

An arithmetic encoder can reach
 this rate, to be compared to
 $H(X, Y)/2 = 0.511$ bpp



$$P(\square|\square) = 94.1\%$$

$$P(\blacksquare|\square) = 5.9\%$$

$$P(\square|\blacksquare) = 33.3\%$$

$$P(\blacksquare|\blacksquare) = 66.7\%$$



Block Coding vs. Context-Based Coding

Comparing the compression efficiency for the "T-shape" image:

Method	Theoretical Limit (bpp)	Practical Rate (bpp)
Symbol-wise ($K = 1$)	$H(X) = 0.586$	1.000 (Huffman)
Block Coding ($K = 2$)	$\frac{H(X^2)}{2} = 0.511$	0.650 (Huffman)
Block Coding ($K = 4$)	$\frac{H(X^4)}{4} = 0.383$	0.433 (Huffman)
Context-Based (1D)	$H(X_n X_{n-1}) = 0.406$	$\approx$ 0.406 (Arithmetic)

With a simple 1D context (the previous pixel) we achieve almost the same performance of a more complicated 2×2 block-based coding
 More complex context can better capture long-term dependencies than using large block sizes.



Block Coding vs. Context-Based Coding

The Engineer's Challenge: Finding the Right Context

Instead of increasing the block size K indefinitely, it is more efficient to use **context-based coding**.

- ▶ If the context captures the "significant past" (e.g., the Markov order N), we reach the entropy rate $\mathcal{H}$ without massive blocks.
- ▶ **Too large context:** Sparse data makes probability estimation unreliable.
- ▶ **Too small context:** Fails to capture the dependencies, leaving redundancy.



Arithmetic coding: conclusions

Advantages

- ▶ It allows encoding long sequences of symbols with linear complexity
- ▶ Block coding: it benefits from high order dependencies and removes the penalty due to non-dyadic distributions
- ▶ Context-based coding: simple model for high order statistics
- ▶ Adaptivity: non-stationary sources



Arithmetic coding: conclusions

Disadvantages and difficulties

- ▶ Implementation may be tricky
- ▶ Need of initialization
- ▶ How to choose contexts?
- ▶ Adaptivity: robust estimation demands for a potentially large training set



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques

Neural Lossless Coding

Conclusions



Exponential-Golomb coding

- ▶ No initialization needed (universal coding)
- ▶ Used for encoding integers (signed or unsigned)
- ▶ No need to decide in advance the minimum and maximum value to encode: any number can be encoded
- ▶ Smaller numbers (in absolute value) get shorter codewords



Exp-Golomb coding: Unsigned Integers

Encoder

1. $n = 0$ is encoded as "1"
2. For $n \in \{1, 2, \dots\}$, write $n + 1$ down in binary using the minimum number of bits:
 $b = \lfloor \log_2(n + 1) \rfloor + 1$.
 Place $b - 1$ zeros before the obtained bitstring for n

Decoder: while there are bits in the bitstream

1. If the next bit is "1", decode $n = 0$.
2. Otherwise, count the number b of consecutive zeros. Read the bitstring of $b + 1$ bits, convert from binary to decimal and remove the offset (-1)

Unsigned EG Coding table

n	$n+1$ on b bits	Leading zeros	$c_U(n)$
0	1	-	1
1	10	0	010
2	11	0	011
3	100	00	00100
4	101	00	00101
5	110	00	00110
6	111	00	00111
7	1000	000	0001000
...	...	...	...



Exp-Golomb coding: Signed Integers

Encoder

1. Map $n \in \mathbb{Z}$ into $m(n) \in \mathbb{N}$:
 - ▶ If $n > 0$, then $m(n) = 2n - 1$
 - ▶ If $n \leq 0$, then $m(n) = -2n$
2. Encode $m(n)$ with the unsigned Exp-Golomb method

Decoder

1. Decode a positive number m with the unsigned Exp-Golomb method
2. Map it to the integers:
 - ▶ If m is odd, $n = \frac{m+1}{2}$
 - ▶ If m is even, $n = -\frac{m}{2}$

Signed EG Coding table

n	$m(n)$	$c_S(n) = c_U(m(n))$
0	0	1
1	1	010
-1	2	011
2	3	00100
-2	4	00101
3	5	00110
-3	6	00111
4	7	0001000
...	...	...

Go to [wooclap.com](https://www.wooclap.com) with the code **GYKKIN**



Principles of Dictionary-Based Coding

Core Ideas

- We give up using pre-established probabilities
- We learn the symbol long-term dependencies during the encoding
- We use a dictionary of long, learned patterns of symbols

Dictionary-based coding stems from the works of Lempel and Ziv (LZ77 and LZ78) and Lempel, Ziv and Welsch (LZW).

The Asymptotic Optimality Theorem

For any **stationary** and **ergodic** source, the dictionary-based coding algorithm is **asymptotically optimal**: as the sequence length n approaches infinity, the average code length L_n converges to the **Entropy Rate** $\mathcal{H}$ of the source



Principles of Dictionary-Based Coding

Key Characteristics:

- ▶ **Dynamic Dictionary Construction:** The dictionary is built on-the-fly during the encoding process
- ▶ **Universality:** No prior knowledge of source statistics or probability distributions is required
- ▶ **Adaptivity:** It automatically tracks and adapts to non-stationary signals where statistics change over time
- ▶ **Practical Efficiency:** It is the fundamental principle behind ubiquitous formats like ZIP, GZIP, etc.
- ▶ **Why use it?** While Entropy coding (Huffman, arithmetic) struggles with very high-order dependencies (large K), dictionary-based methods scale efficiently by identifying long redundant patterns



Main Dictionary-Based Algorithms

Dictionary coding methods are generally classified into two generations: the sliding window approach (LZ77 family) and the explicit dictionary approach (LZ8 family).

Algorithm	Year	Technical Characteristic
LZ77	1977	Uses a sliding window to reference previous strings via (offset, length) pairs.
LZ78	1978	Builds an explicit dictionary of strings encountered in the data stream.
LZSS	1982	Enhances LZ77 by using a flag bit to distinguish between literals and pointers.
LZW	1984	A LZ78 refinement that eliminates character transmission by pre-filling the dictionary.
LZMA	1998	Combines large dictionary sizes with range coding (arithmetic) for high compression.



LZW Encoding: Dictionary-Based Logic

The Greedy Matching Process

- ▶ The encoder reads the input stream symbol by symbol.
- ▶ For each new input, it checks if the current pattern (Prefix W + Next K) already exists in the dictionary.
- ▶ **Iterative Search:** If $W + K$ is found, the encoder extends the search by reading the next symbol, continuing until a **new pattern** is encountered.

Dictionary Evolution and Merging

- ▶ Once a new pattern $W + K$ is identified:
 1. The index of the longest known prefix (W) is sent to the bitstream.
 2. The new combined pattern ($W + K$) is added to the dictionary.
- ▶ **Prefix Merging:** This mechanism merges known prefixes with new suffixes, allowing the dictionary to naturally evolve and "track" local statistics.



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Output



Dictionary-based coding: example (LZW)

Input

0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

Output

0 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Output



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

0

Output



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 X

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

0

Output

0



Dictionary-based coding: example (LZW)

Input

0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0
0

Output

0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0
0

Output

0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

0

Output

0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

1

Output

0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 0 X

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

1

Output

0 2



Dictionary-based coding: example (LZW)

Input

0 0 0 **1** 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 **1** 0 0
 0 0
 1

Output

0 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 1 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

1

Output

0 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 1 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 0

1

Output

0 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 1 ~~X~~

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 0

1

Output

0 2 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
 0 0 0
 1

Output

0 2 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
0 0 0
1

Output

0 2 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 0

1

Output

0 2 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

0 0 0 0

1 0

Output

0 2 1



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 0 ~~X~~

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

0 0 0 0

1 0

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0
		0	0	0	0
			1		0

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

Prefix

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1
		0	0	0
		0	1	

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1
0 0 0
0 1

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1
		0	0	0
		0	1	

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1
		0	0	0
		0	1	

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

New 0 0 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 0

0 1

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 0 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

0 0 0 0

0 1 0

0

Output

0 2 1 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 0 0 ~~X~~

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

0 0 0 0

0 1 0

0

Output

0 2 1 2 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0** 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0
	0	0	0	0	
	0	1		0	
				0	

Output

0 2 1 2 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

New 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

0 0 0 0

0 1 0

0

Output

0 2 1 2 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Add 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 0

0 0 0 0 1

0 1 0

0

Output

0 2 1 2 2



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Output 0 ~~X~~

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 0

0 0 0 0 1

0 1 0

0

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0	0
		0	0	0	0	1
		0	1	0		
				0		

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 **0 1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Prefix 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0										
1		0	0	0	0										
		0	1												

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 **1** 0 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 1

Table

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E
0	1	0	0	1	0									
1		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0** 1 0 0 0 0 1 0 0 0 0 0 1 0

Present 1 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0									
1		0	0	0	0									
		0	1		0									
					0									

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0 1** 0 0 0 0 1 0 0 0 0 0 1 0

New 1 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

1 0 0 0 0

0 1 0

0

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0 1** 0 0 0 0 1 0 0 0 0 0 1 0

Add 1 0 1

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 1

1 0 0 0 0 0

0 1 0 1

0

Output

0 2 1 2 2 0



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 0 **1 0 1** 0 0 0 0 1 0 0 0 0 0 1 0

Output 1 0 X

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 1

1 0 0 0 0 0

0 1 0 1

0

Output

0 2 1 2 2 0 4



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

New 1 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 1

1 0 0 0 0 0

0 1 0 1

0

Output

0 2 1 2 2 0 4



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

Output 1 0 ~~X~~

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0 1 1

1 0 0 0 0 0 0

0 1 0 1 0

0

Output

0 2 1 2 2 0 4 4



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

Prefix 1 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0	1	1
1	0	0	0	0	0	0	0
	0	1		0	1	0	
				0			

Output

0 2 1 2 2 0 4 4



Dictionary-based coding: example (LZW)

Input

0 0 0 1 0 0 0 0 0 0 1 0 **1 0 0** 0 0 1 0 0 0 0 0 1 0

Prefix 1 0 0

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0	1									
1		0	0	0	0	0									
		0	1	0	0	1									

0

Output

0 2 1 2 2 0 4 4



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Input

0 2 1 2 2 0

New

Output



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1

Input

0 2 1 2 2 0

New

2 0x

Output

0x



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

x

New

2 0x

Input

0 2 1 2 2 0

Output

0x



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0

x

New

3 0xy

Input

0 2 1 2 2 0

Output

0x

00xy



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

y

New

3 00y

Input

0 2 1 2 2 0

Output

000y



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0

0 0

y

New

4 1z

Input

0 2 1 2 2 0

Output

000y

0001z



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 z

1

New

4 1z

Input

0 2 1 2 2 0

Output

0001z



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1

0 0 z

1

New

5 00w

Input

0 2 1 2 2 0

Output

0001z

000100w



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1 0 0 1 0

 0 0 0 0

 1 w

New

5 00w

Input

0 2 1 2 2 0

Output

000100w



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0	1	0	0	1	0
		0	0	0	0
			1		w

New

5 00w

Input

0 2 1 2 2 0

Output

000100w



Dictionary-based decoding: example (LZW)

Table

0 1 2 3 4 5 6 7 8 9 A B C D E

0 1	0 0	1
	0 0	0
	0 1	

New

2 000

Input

0 2 1 2 2 0

Output

0001000



Dictionary Coding: Advanced Insights

The LZW Patent Controversy

- ▶ GIF used **LZW**, which was patented by Unisys.
- ▶ In the late 90s, Unisys began enforcing royalties for software developers.
- ▶ **PNG** was developed as a "free" alternative, choosing **LZ77 (Deflate)** specifically to bypass these legal constraints.

Modern Web (Brotli & Zstd)

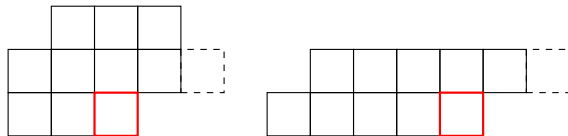
- ▶ Modern algorithms like **Brotli** use a **static dictionary**.
- ▶ This dictionary contains common strings found in web traffic (HTML tags, JS keywords).
- ▶ *Pre-warming* the dictionary allows for high compression even on very small files.



JBIG-1

Joint Bi-level Image Experts Group

- ▶ Standard ISO/IEC 11544 and recommendation ITU-T T.82
- ▶ Context-based arithmetic coding
- ▶ Progressive coding (resolution scalability)
- ▶ *Template* of 10 pixels, 2 shapes with a single variable pixel position



Two rows template: faster but $\approx 5\%$ rate increase

Joint Bi-level Image Experts Group

- ▶ Standard ISO/IEC 14492 and recommendation ITU-T T.88
- ▶ Image segmented into **text**, **halftones**, and **other**
- ▶ **Text**: A dictionary of symbols is created and used for coding
- ▶ **Halftone**: the gray-scale image is encoded using a dictionary of *halftone patterns*
- ▶ **Other**: Context-based arithmetic coding
- ▶ PDF files (version 1.4 and behind) can contain JBIG2 data



Lossless Image Compression: JPEG-LS

Unlike general data, images possess strong **spatial correlation**. JPEG-LS (based on the LOCO-I algorithm) is the standard for low-complexity lossless compression.

The Predictive Framework

- Uses a causal neighborhood (A, B, C) to predict the current pixel X .

C	B
A	X

C	B
A	x

- ▶ **MED Predictor** (Median Edge Detector):

$$\hat{x} = \begin{cases} \min(A, B) & \text{if } C \geq \max(A, B) \\ \max(A, B) & \text{if } C \leq \min(A, B) \\ A + B - C & \text{otherwise} \end{cases}$$

- Specifically designed to handle sharp edges without overshoot.

Encoding the Residual

- ▶ **Context Modeling:** The prediction error $e = x - \hat{x}$ is refined based on local gradients
- ▶ **Golomb-Rice Coding:** Used to encode the residuals
- ▶ **Run Mode:** Switches to run-length encoding in areas with identical pixels



Lossless Image Compression: PNG vs. JPEG-LS

Comparison: PNG vs. JPEG-LS

- ▶ **PNG:** Uses LZ77 (Dictionary) + Huffman. Better for computer graphics, icons, and text-heavy images. Can use a pixel predictor similar to JPEG-LS
- ▶ **JPEG-LS:** Predictive. Better for natural photographs and medical imaging where local gradients follow a statistical model.

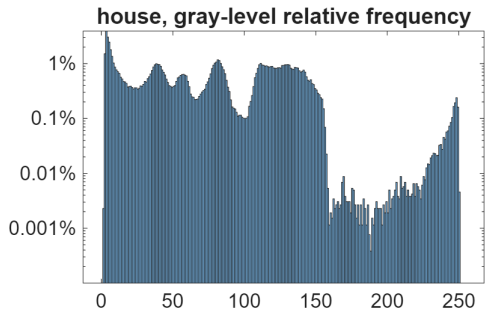


Lossless Compression: Visual Analysis

House (PGM), 8 bits per pixel



Luminance Histogram, $H = 7.06$ bpp



Distribution is spread across the 0–255 range. High entropy confirms that raw pixel values are not concentrated.



Lossless Compression: Baseline Analysis

The experiment evaluates different lossless techniques on a 256-level gray-scale image (house .pgm, 8 bpp).

Pixel-level Statistics:

- ▶ **Entropy (H):** Estimated via relative frequency of gray levels. For the "house" image, $H = 7.056$ bpp.
- ▶ **Huffman Coding:** Results in an average length of 7.081 bpp.
- ▶ **ZIP (Deflate, i.e. LZ77):** Achieves 4.003 bpp. It outperforms pixel-wise Huffman by exploiting string repetitions.
- ▶ **Unsigned Exp-Golomb:** Results in 11.320 bpp (inefficient for raw luminance).

Observation

Huffman coding at the pixel level is close to the entropy but offers poor compression because it ignores spatial dependencies. ZIP performs better as it identifies block redundancies



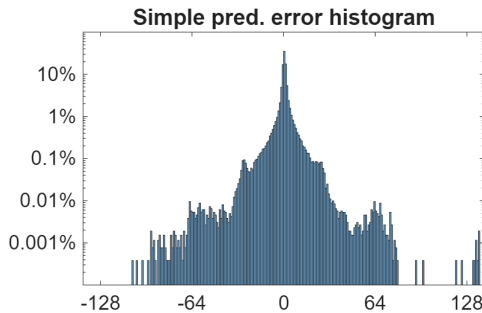
DPCM: 1D (horizontal) Spatial Prediction

To exploit spatial correlation, we define a predictor $p(n)$ based on the previous pixel:

$$p(n) = x(n-1) \implies \text{Error: } y(n) = x(n) - x(n-1)$$

Sample prediction is often referred to as **DPCM** (for Differential Pulse Coding Modulation)

- ▶ **Residual Entropy:** 3.312 bpp.
- ▶ **Signed Exp-Golomb:** 3.428 bpp.
- ▶ **Huffman:** 3.383 bpp.
- ▶ **ZIP:** 3.230 bpp.



The signal is now less correlated, and the error is centered at zero.



DPCM: 2D Spatial Prediction

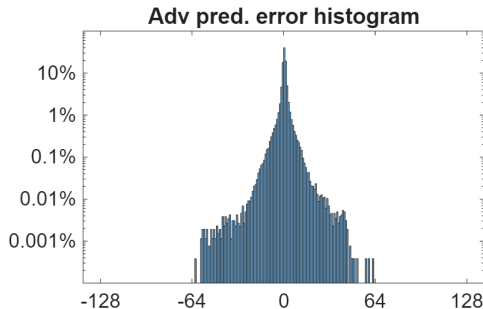
A better predictor considers a causal neighborhood (A, B, C) for pixel X:

C	B
A	X

The best prediction direction is determined comparing $|B - C|$ with $|A - C|$: if the former is smaller than the latter, horizontal prediction is considered more reliable

Performance on "house":

- ▶ **Residual Entropy:** 2.830 bpp.
- ▶ **Huffman:** 2.894 bpp.
- ▶ **Signed Exp-Golomb:** 2.941 bpp.
- ▶ **ZIP:** 3.141 bpp.

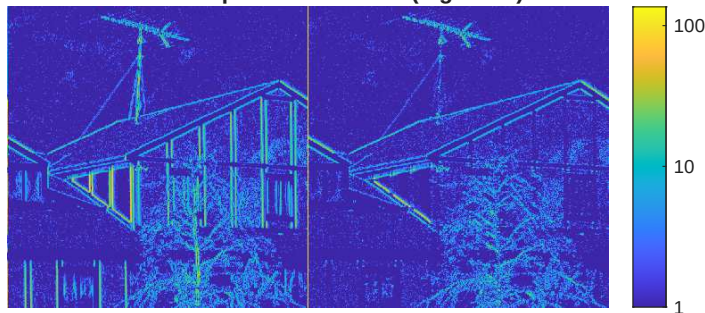


Note how large errors are much less common with respect to the 1D predictor



Visual comparison of predictors

Abs of the prediction errors (log scale)



The 1D predictor (left) removes horizontal structures.
 The 2D predictor is adaptively capable of following horizontal and vertical structures, but struggles with diagonal ones.
 More effective prediction strategies can be handcrafted or learned



Summary of Results (House Image)

Method	Direct Image	Simple Pred.	Adv. Pred.
Entropy	7.06	3.31	2.83
Huffman	7.08	3.38	2.89
Exp-Golomb	11.32	3.43	2.94
ZIP	4.00	3.23	3.14

Key Conclusions:

- ▶ Prediction is very effective for natural images: entropy goes from ~ 7 to ~ 2.8 bpp in this example. For other images the residual entropy is often around 4 bpp
- ▶ DEFLATE (dictionary) only is efficient for direct coding (pattern matching). For residuals, simple entropy coders (Huffman, EG) are equivalent and less complex, in particular EG offers a great complexity/performance trade-off for sparse residuals
- ▶ Popular formats like PNG use a combination of all these tools



Outline

Introduction

Principles of Information Theory

Optimal coding

Other Techniques

Neural Lossless Coding

Conclusions



Neural Lossless Coding: The Density Estimation Perspective

Neural Network (NN) estimates the true, high-order distribution $P(X^n)$ of the source

The Information-Theoretic Limit

In Neural Lossless Coding (NLC), we minimize the **Cross-Entropy** between the true distribution P and the neural model (estimated distribution) Q :

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

- ▶ $H(P)$ is the source entropy (physical limit).
- ▶ D_{KL} is the penalty for model inaccuracy (redundancy).
- ▶ The NN does not "compress" directly, but acts as a **probability distribution estimator** providing Q to an arithmetic coder.



Overcoming the Curse of Dimensionality

- ▶ **Traditional Approach:** State-space explodes exponentially (M^N). A 3-pixel context at 8 bpp requires ≈ 16.7 million entries .
- ▶ **Neural Approach:** Maps contexts to a continuous vector space. It learns to group similar patterns, generalizing even to unseen contexts.
- ▶ **Efficiency:** While classical context-coding reached 0.406 bpp for the T-shape, NLC aims for the true entropy rate $\mathcal{H}$ by leveraging global dependencies (Attention/Transformers).

Key takeaway: NNs replace look-up tables with non-linear function approximation.



1. Autoregressive Models: Causal Density Estimation

Mathematical Foundation

These models treat the image as a sequence of random variables and apply the **Chain Rule** of probability:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid x_1, \dots, x_{i-1})$$

- ▶ **Working Principle:** The network estimates the conditional PDF of the current pixel x_i given all previous pixels (causal context).
- ▶ **Information Theory:** Since the model captures high-order dependencies, the bit-rate approaches the **Entropy Rate** $\mathcal{H}$.
- ▶ **Trade-off:**



2. Latent Variable Models: Mapping to Manifolds

Theoretical Framework

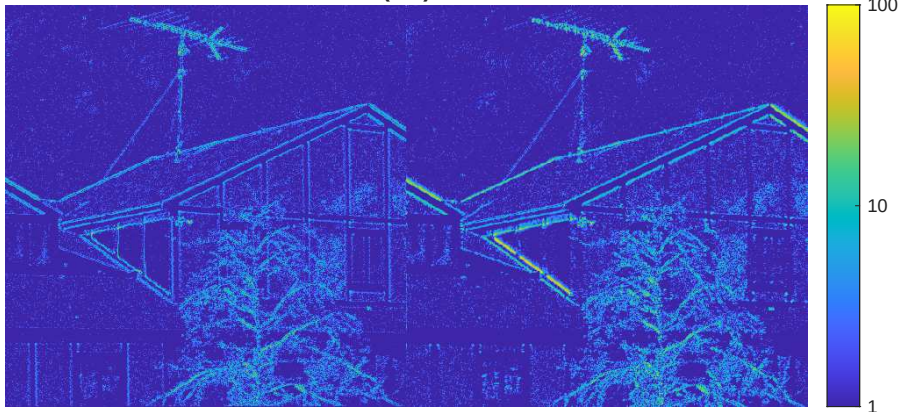
Based on **Variational Inference**. The image x is mapped to a lower-dimensional latent representation z (similar to a non-linear PCA).

- ▶ **Two-stage Coding:**
 1. Encode the latent features z (the "summary").
 2. Encode the data x given z (the "residuals" or likelihood).
- ▶ **The Bits-back Principle:** In a sequence of images, the bits used to specify z can be "recovered" from the randomness of the previous message.
- ▶ **Limit:** The compression performance is bounded by the **ELBO** (Evidence Lower Bound), which is the variational approximation of the true log-likelihood.



Experimental Results: Residual Analysis

Neural residual (left) vs. 2D Predictor





Experimental Results: Residual Analysis

Neural vs. 2D Predictor

- ▶ **Left (Neural):** The residual map is noticeably "cleaner" and more uniform.
- ▶ **Right (2D):** Shows stronger residuals (yellow/green) especially on diagonal structures and the antenna.
- ▶ **Key Observation:** The NN better handles complex textures and non-linear transitions that the deterministic logic ($|C - B| < |C - A|$) misses.



Counting the Bitrate Overhead

The total cost to transmit the image is:

$$R_{total} = R_{residuals} + \frac{L_{model}}{N} \quad [\text{bits/pixel}]$$

- ▶ $L_{model} = (\text{Total Parameters}) \times (\text{Precision})$.
- ▶ **Example:** A small [10, 5] MLP with 3 inputs has ≈ 100 weights. Using 32-bit floats, $L_{model} = 3200$ bits.
- ▶ Total rate: $R = 2.704 + \frac{3200}{512^2} \approx 2.716$
- ▶ **Trade-off:** For small images (N), the model cost can outweigh the compression gain. NLC is more efficient for large datasets (weight shared among all the images) or high-resolution images.



Current Limitations of Neural Lossless Coding

Despite superior compression ratios, NLC faces significant deployment hurdles.

- ▶ **Computational Complexity:** AR models require N forward passes for an N -pixel image, making real-time decoding nearly impossible without massive optimization.
- ▶ **Generalization Gap:** NNs are sensitive to "out-of-distribution" data. A model trained on natural images may fail (or expand) medical or satellite imagery.
- ▶ **Hardware Requirements:** Unlike Huffman or LZW, NLC requires GPUs or specialized AI accelerators (NPUs) at the decoder side.
- ▶ **Deterministic Requirements:** Lossless coding requires perfect bit-reproducibility across different hardware (handling floating-point rounding errors).



SOTA Comparison: Neural vs. Traditional Lossless

Performance comparison on standard datasets (e.g., Kodak). Values in **bits per pixel (bpp)**: lower is better.

Codec	Type	Avg. bpp	Decoding Complexity
PNG (Deflate)	Traditional	4.30 – 4.50	Very Low
WebP (Lossless)	Traditional	3.70 – 3.90	Low
JPEG-LS	Traditional	3.60 – 3.80	Low
L3C (CNN)	Neural (AR)	3.30 – 3.50	Very High
MS-ILLC	Neural (Hier.)	3.10 – 3.30	High
Transformer-based	Neural (Att.)	2.90 – 3.10	Ultra High

- **The Efficiency Gap:** Neural models provide a **15% to 30%** improvement over PNG.
- **Contextual Power:** Transformers can identify repeating textures or semantic patterns



Comparison of Lossless Coding Techniques

Method	Efficiency	Complexity	Latency	Best use case
Neural (SOTA)	Highest	Ultra High	High	SOTA Image/Video
Arithmetic	High (to H)	High (CPU)	Med	Bi-level images, CABAC
Dictionary	Universal	Moderate	Low/Med	Repeating patterns
Huffman	Good to high ($1 < L < H + 1$)	Low/Med	Very Low	JPEG, general purpose
Exp-Golomb	Good (approx.)	Very Low	Negligible	Metadata, Residuals

Statistical Modeling

- ▶ **Huffman/Arithmetic:** Use probabilities $P(X)$.
- ▶ **Neural:** Estimates $P(X)$ via non-linear density estimation.

The 1-bit Penalty

- ▶ **Huffman:** Fails if $H(X) < 1$ bit/symbol
- ▶ **Arithmetic:** Efficient below 1 bit/symbol



Context-Based vs. Block Coding

Experimental Data (T-shape image)

Method	Limit (bpp)	Practical (bpp)
Symbol-wise ($K = 1$)	0.586	1.000 (Huffman)
Block Coding ($K = 4$)	0.383	0.433 (Huffman)
Context-Based	0.406	$\approx$ 0.406 (Arithmetic)

- ▶ **Block Coding:** Complexity grows exponentially (M^K)
- ▶ **Context-Based:** Reaches the entropy rate $\mathcal{H}$ by capturing the "significant past" without massive tables.

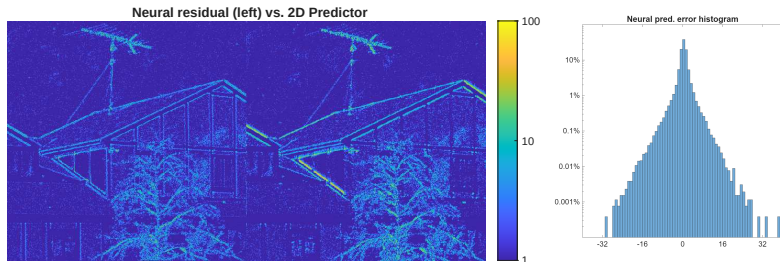


Neural Lossless Coding: SOTA Performance

Density Estimation Perspective

- ▶ NNs minimize the **Cross-Entropy** between the true source P and the model Q .
- ▶ They allows to have better predictors.

Experimental Validation (house.pgm):



The Neural Predictor (2.70 bpp) outperforms the 2D Predictor (2.83 bpp).



Lossless Design Matrix

If the source is...	Then use...	Why?
Memoryless	Huffman / Arithmetic	Respects $H(X)$ bound
Stationary w/ Memory	Context-Adaptive	Optimal for Markov stats
Locally Stationary	Adaptive Arithmetic	Tracks changing statistics
Highly Complex	Neural Models	Captures non-linear dependencies

Theoretical Convergence

All methods aim for $L \approx \mathcal{H}$. Choice depends on **complexity**, **latency**, and **model cost** (transmitting the NN weights).



Final Takeaway

Use **Huffman/EG** for speed, **Arithmetic/Context** for high compression, **Dictionary** for universal files, and **Neural Models** for the ultimate SOTA performance.